
WinSports

Plataforma de Analítica de Streaming Deportivo

Procesamiento de eventos de reproducción sobre MongoDB Atlas,
API en FastAPI y dashboard en React

Informe Final

Ingeniería de Datos

Autor: Juan José Castillo

Stack: MongoDB Atlas · FastAPI · React 19

Versión: 0.1.0

Fecha: Mayo de 2026

Índice

1. Resumen	3
2. Introducción	3
2.1. Contexto y motivación	3
2.2. Objetivos	3
3. Metodología y requisitos	4
3.1. Enfoque de desarrollo	4
3.2. Requisitos funcionales	4
3.3. Requisitos no funcionales	5
4. Arquitectura del sistema	5
4.1. Stack tecnológico	5
4.2. Justificación: NoSQL frente a modelo relacional	5
4.3. Separación de capas	6
4.4. Flujo de datos general	6
5. Conjunto de datos	6
5.1. Origen y volumen	6
5.2. Diccionario de datos	7
5.3. Catálogo de tipos de evento	7
5.4. Ejemplo de filas crudas	8
6. Modelo de datos	8
6.1. Colección <code>events</code> — fuente de verdad	8
6.2. Colección <code>sessions</code> — derivada de <code>events</code>	9
6.3. Colección <code>content_stats</code> — derivada de <code>sessions</code>	10
6.4. Estrategia de índices	11
7. Ingesta y pipeline de datos	12
7.1. Parseo e ingesta de CSV	12
7.2. Construcción de sesiones	12
7.3. Agregación de <code>content_stats</code>	13
7.4. Idempotencia mediante <code>upsert</code>	13
8. API REST	13
8.1. Módulo Overview	13
8.2. Módulo QoE	14
8.3. Módulo Users	14
8.4. Módulo Admin	14
8.5. Ejemplos de petición y respuesta	14
8.6. Manejo de errores	15

9. Dashboard (Frontend)	15
9.1. Páginas	16
9.2. Componentes y temas	16
9.3. Mapa de usuarios: jitter determinístico	16
10.Sistema de alertas	16
11.Panel de administración	17
12.Decisiones de diseño	17
13.Escalabilidad y requisitos no funcionales	18
14.Pruebas	19
14.1. Backend	19
14.2. Frontend	19
15.Riesgos y limitaciones	19
15.1. Resueltos	19
15.2. Abiertos / a vigilar	19
16.Trabajo futuro	20
17.Conclusiones	20
18.Glosario	20
19.Referencias	21
A. Anexo: puesta en marcha	22
B. Anexo: estructura del repositorio	22

1 Resumen

WinSports es un sistema de analítica construido sobre los eventos de reproducción de **Win Sports**, plataforma colombiana de streaming deportivo. El sistema ingiere registros crudos de eventos (**START**, **BUFFERING**, **PAUSE**, **COMPLETE**, etc.) desde archivos CSV hacia **MongoDB Atlas**, los transforma en colecciones derivadas mediante un pipeline de agregación, y los expone a través de una **API REST en FastAPI** consumida por un **dashboard interactivo en React**.

El núcleo del diseño es un modelo de datos de tres niveles (**events** → **sessions** → **content_stats**) que separa la fuente de verdad inmutable de las vistas precalculadas que alimentan el tablero. Sobre esa base, el dashboard ofrece cuatro perspectivas analíticas —*Overview*, *QoE* (calidad de experiencia), *Usuarios* y *Admin*—, además de un módulo de *Alertas* operativas. El proyecto incluye una suite de pruebas en backend (pytest con Mongo en memoria) y frontend (Vitest + Testing Library).

2 Introducción

2.1 Contexto y motivación

Cada vez que un usuario reproduce contenido en una plataforma de streaming, el reproductor emite una secuencia de eventos: el inicio de la reproducción, los episodios de *buffering*, las pausas, los hitos de progreso (primer cuartil, mitad, tercer cuartil) y la finalización. Analizar este flujo permite responder preguntas de negocio clave: ¿qué contenido se consume más?, ¿qué tan buena es la calidad de reproducción?, ¿en qué punto los usuarios abandonan un contenido?, ¿qué dispositivos predominan?

Estos datos presentan dos características que condicionan el diseño:

- **Esquema flexible.** No todos los eventos traen los mismos campos: **buffer_time** solo aparece en eventos de buffering; **episode** y **season** solo en series. Forzar un esquema rígido relacional generaría una proliferación de valores nulos.
- **Alto volumen.** En producción, los eventos llegarían como flujo en vivo, alcanzando un orden de magnitud de $\sim 100\text{M}$ de documentos. La arquitectura debe escalar horizontalmente sin rediseñarse.

Por estas razones se eligió **MongoDB**, una base de datos documental que maneja naturalmente el esquema flexible y escala horizontalmente vía *sharding*.

2.2 Objetivos

1. Diseñar un modelo de datos documental que separe la captura cruda de eventos de las métricas analíticas precalculadas.
2. Implementar un pipeline reproducible que transforme un CSV de eventos en colecciones derivadas listas para consulta.

3. Exponer las métricas mediante una API REST con contratos bien definidos y validados.
4. Construir un dashboard que comunique las métricas de forma visual e interactiva a un equipo no técnico.
5. Garantizar la idempotencia de la carga y la corrección mediante pruebas automatizadas.

3 Metodología y requisitos

3.1 Enfoque de desarrollo

El sistema se construyó de forma **iterativa e incremental**, en cortes verticales: cada métrica del dashboard se llevó de extremo a extremo (índice y consulta en la base de datos → repositorio → esquema y ruta de la API → componente de React) antes de pasar a la siguiente. Este enfoque mantuvo el sistema funcional en todo momento y permitió validar el contrato entre capas de forma temprana. El control de versiones se gestionó con Git, y las dependencias con `uv` (backend) y `npm` (frontend).

3.2 Requisitos funcionales

ID	Requisito
RF1	Ingerir un CSV de eventos y persistirlo en la colección <code>events</code> .
RF2	Derivar automáticamente <code>sessions</code> y <code>content_stats</code> a partir de los eventos.
RF3	Exponer métricas de portada (reproducciones, usuarios, contenido, dispositivos, mapa).
RF4	Exponer métricas de calidad de experiencia (buffer, re-buffering, tiempo de inicio).
RF5	Exponer métricas de comportamiento (funnel, heatmap, perfiles, completitud).
RF6	Detectar y reportar alertas operativas por severidad.
RF7	Permitir la gestión de datos (carga, conteo, edición y borrado) desde un panel.

3.3 Requisitos no funcionales

ID	Requisito
RNF1	Idempotencia: re-cargar el mismo CSV no debe duplicar datos.
RNF2	Escalabilidad horizontal hacia el orden de $\sim 100M$ de documentos.
RNF3	Tolerancia a fallos en la carga masiva (un documento inválido no detiene el lote).
RNF4	Modularidad: la API nunca accede a Mongo directamente; el front solo consume REST.
RNF5	Privacidad: no exponer identificadores completos de suscriptor en respuestas públicas.
RNF6	Respuesta interactiva: consultas del dashboard servidas desde vistas precalculadas.
RNF7	Verificabilidad: cobertura de pruebas en backend y frontend.

4 Arquitectura del sistema

El sistema se organiza en capas con responsabilidades estrictamente separadas. La API nunca accede a MongoDB directamente: toda consulta pasa por la capa de repositorios. El frontend solo consume la API REST. Esta modularidad permite que base de datos, API y frontend evolucionen de forma independiente.

4.1 Stack tecnológico

Capa	Tecnología
Base de datos	MongoDB Atlas + Motor (driver asíncrono)
API	FastAPI + Uvicorn + Pydantic v2
Frontend	React 19 + Vite + Recharts + Leaflet
Entorno backend	Python 3.14 + uv
Entorno frontend	Node.js 18+
Pruebas	pytest + mongomock-motor · Vitest + Testing Library

4.2 Justificación: NoSQL frente a modelo relacional

La elección de una base de datos documental no es arbitraria; responde a la naturaleza del dato y a los requisitos no funcionales:

Criterio	Relacional (SQL)	Documental (MongoDB)
Esquema	Rígido; campos opcionales obligan a tablas dispersas o muchos NULL.	Flexible; cada evento lleva solo los campos que aplican.
Escalado	Vertical, principalmente; <i>sharding</i> complejo.	Horizontal nativo vía <i>sharding</i> .
Lectura analítica	<i>Joins</i> costosos sobre tablas grandes.	Vistas materializadas (colecciones derivadas) sin <i>joins</i> .
Ingesta masiva	Validación estricta detiene lotes ante un error.	<code>validationAction: warn</code> permite degradar con gracia.
Sincronización	Requiere triggers / ETL externo.	<i>Change streams</i> nativos.

4.3 Separación de capas

La estructura del repositorio refleja directamente la arquitectura:

```

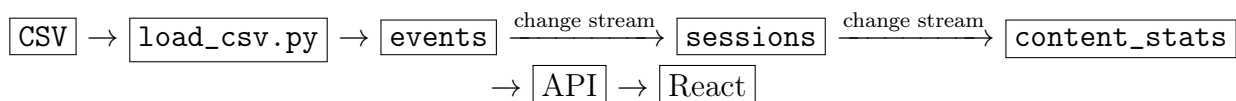
1 db/collections/   validators + indices + setup de cada coleccion
2 db/indexes/       definicion de indices (importados por collections)
3 db/pipelines/     logica de construccion de colecciones derivadas
4 db/repositories/  queries hacia Mongo (una funcion por operacion)
5 db/csv_ingest.py  parseo/ingesta de CSV (compartido por script y admin)
6 api/schemas/     modelos Pydantic - forma del JSON que recibe React
7 api/routes/       endpoints FastAPI - une repositories con schemas
8 config/           settings (.env) + constantes (colecciones, centroides)
9 frontend/         React + Vite (pages, components, layout, styles, test)

```

Listing 1: Organización por responsabilidades

4.4 Flujo de datos general

El dato recorre el sistema en un solo sentido, de la fuente cruda hacia las vistas agregadas:



En **desarrollo**, `load_csv.py` inserta los eventos y `scripts/test_pipeline.py` construye manualmente las colecciones derivadas. En **producción**, los *change streams* de MongoDB mantienen `sessions` y `content_stats` sincronizadas en tiempo real ante cada nuevo evento.

5 Conjunto de datos

5.1 Origen y volumen

La fuente es un CSV de eventos de reproducción de Win Sports. El repositorio incluye dos archivos de muestra para desarrollo y pruebas: `data/50_data.csv` (50 eventos) y

data/1000_data.csv (1.000 eventos). El pipeline se validó además con cargas del orden de **cientos de miles de eventos** (~300k), confirmando que el modelo de colecciones derivadas sostiene el dashboard sin degradar los tiempos de respuesta.

5.2 Diccionario de datos

El CSV original trae 22 columnas. La función `parse_row` en `db/csv_ingest.py` las mapea a los campos del documento de `events`, casteando cada una al `bsonType` esperado:

Columna CSV	Campo evento	Tipo
Date	date	date
CountryCode	country_code	string
SubscriberID	subscriber_id	string
CustomerId	customer_id	string
ContentType	content_type	string
Title	title	string
Episode	episode	int
SeriesTitle	series_title	string
ReleaseYear	release_year	int
Duration	duration	int
Season	season	int
Genres	genres	string
DeviceType	device_type	string (enum)
TypeEvent	type_event	string
Position	position	int
Language	language	string
Bitrate	bitrate	int
BufferTime	buffer_time	double
PlaybackNetTime	playback_net_time	int
deviceDescription	device_description	string
CALC_ProgramType	calc_program_type	string
CALC_BitrateType	calc_bitrate_type	string

5.3 Catálogo de tipos de evento

El campo `type_event` dirige toda la lógica de negocio. Los tipos relevantes son:

Tipo	Significado
START	Inicia una reproducción (delimita el comienzo de una sesión).
START-BUFFERING	Buffer previo al primer frame (mide el <i>startup time</i>).
RE-BUFFERING	Interrupción por buffer durante la reproducción.
PAUSE	Pausa del usuario.
FIRSTQUARTILE	Se alcanzó el 25 % del contenido.
MIDPOINT	Se alcanzó el 50 % del contenido.
THIRDQUARTILE	Se alcanzó el 75 % del contenido.
COMPLETE	El contenido se vio hasta el final.

5.4 Ejemplo de filas crudas

```

1 Date, CountryCode, SubscriberID, CustomerId, ..., DeviceType, TypeEvent, Position
  ..., BufferTime, ...
2 2026-01-01 08:58:00, C0, 0d16...20a9, 6955...de75, ..., WEB, START, 0, ..., ...
3 2026-01-01 08:58:05, C0, 0d16...20a9, 6955...de75, ..., WEB, START-BUFFERING
  , 0, ..., 6.805, ...

```

Listing 2: Dos eventos del CSV (cabecera abreviada)

Nótese que `BufferTime` solo viene poblado en el evento de buffering: ese es exactamente el tipo de esquema flexible que motiva el uso de MongoDB.

6 Modelo de datos

El modelo se compone de tres colecciones con responsabilidades separadas. Cada colección downstream es una vista materializada de la anterior, lo que evita recalculación de agregaciones costosas en cada petición del dashboard.

6.1 Colección events — fuente de verdad

Almacena un documento por cada evento del CSV. Es un *log* crudo, inmutable: nunca se modifica tras la inserción. Su validador (`$jsonSchema`) exige cinco campos obligatorios y deja el resto opcional, respetando el esquema flexible de los eventos de streaming.

Campo	Descripción
date	Fecha-hora del evento (obligatorio, <code>bsonType: date</code>).
subscriber_id	Identificador del suscriptor (obligatorio).
customer_id	Identificador de la pieza de contenido (obligatorio).
type_event	Tipo de evento: <code>START</code> , <code>PAUSE</code> , <code>COMPLETE</code> , etc. (obligatorio).
device_type	Plataforma: <code>ANDR</code> , <code>IOS</code> o <code>WEB</code> (obligatorio, <code>enum</code>).
buffer_time	Tiempo de buffer; solo en eventos de buffering.
episode, season	Solo en contenido tipo episodio/serie.
position	Posición de reproducción, en segundos.
country_code	Código de país (alimenta el mapa de usuarios).

Validación. El validador usa `validationAction: "warn"` en lugar de `"error"`: durante una carga masiva, un documento con un campo inesperado no debe detener la inserción de millones de filas; el problema se registra sin interrumpir el proceso. Los enteros aceptan `["int", "long"]` porque `bitrate` y `position` pueden superar el rango de `int32` en el dataset completo.

```

1 {
2   "date": ISODate("2026-01-01T08:58:05Z"),
3   "country_code": "CO",
4   "subscriber_id": "0d1687a8...390020a9",
5   "customer_id": "69553fe8c788eafebe63de75",
6   "content_type": "MOVIE",
7   "title": "PARTIDO APARTE",
8   "duration": 3540,
9   "device_type": "WEB",
10  "type_event": "START-BUFFERING",
11  "position": 0,
12  "buffer_time": 6.805,
13  "calc_bitrate_type": "HIGH"
14 }
```

Listing 3: Documento de ejemplo en `events`

6.2 Colección `sessions` — derivada de `events`

Una *sesión* agrupa todos los eventos de un par (`subscriber_id`, `customer_id`) entre un `START` y el siguiente `START`. Precalcula las métricas de progreso y calidad para no recalcularlas sobre el conjunto completo de eventos en cada consulta.

- **Clave única:** `subscriber_id + customer_id + start_time`.
- **Progreso:** `reached_firstquartile / midpoint / thirdquartile / complete` (booleanos del funnel), `completion_pct`, `max_position`.

- **Calidad (QoE):** total_buffer_time, rebuffer_count, pause_count, dominant_bitrate_type.

```

1 {
2   "subscriber_id": "0d1687a8...390020a9",
3   "customer_id": "69553fe8c788eafebe63de75",
4   "start_time": ISODate("2026-01-01T08:58:00Z"),
5   "end_time": ISODate("2026-01-01T09:55:00Z"),
6   "device_type": "WEB",
7   "duration": 3540,
8   "total_buffer_time": 6.805,
9   "rebuffer_count": 0,
10  "pause_count": 1,
11  "max_position": 3120,
12  "completion_pct": 88.14,
13  "reached_firstquartile": true,
14  "reached_midpoint": true,
15  "reached_thirdquartile": true,
16  "reached_complete": false,
17  "event_count": 7
18 }

```

Listing 4: Documento de ejemplo en sessions

6.3 Colección content_stats — derivada de sessions

Un documento por customer_id (pieza de contenido), con métricas agregadas que alimentan directamente los rankings del dashboard. Importante: customer_id representa una pieza concreta (un episodio, una película, un partido), no un programa completo; para métricas por serie se agrupa por series_title.

- **Volumen:** total_plays, unique_viewers.
- **Funnel:** firstquartile_rate, midpoint_rate, thirdquartile_rate, completion_rate.
- **Calidad:** avg_buffer_time, rebuffer_rate, avg_completion_pct.
- **Breakdown:** plays_by_device = {ANDR, IOS, WEB}.

```

1 {
2   "customer_id": "69553fe8c788eafebe63de75",
3   "title": "PARTIDO APARTE",
4   "content_type": "MOVIE",
5   "total_plays": 40,
6   "unique_viewers": 33,
7   "firstquartile_rate": 82.5,
8   "midpoint_rate": 67.5,
9   "thirdquartile_rate": 50.0,
10  "completion_rate": 30.0,
11  "avg_buffer_time": 2.314,
12  "rebuffer_rate": 18.5,
13  "avg_completion_pct": 64.2,

```

```

14 "plays_by_device": { "WEB": 22, "ANDR": 12, "IOS": 6 }
15 }

```

Listing 5: Documento de ejemplo en `content_stats`

6.4 Estrategia de índices

Cada colección define índices alineados con sus patrones de consulta. Por ejemplo, en `events` el índice compuesto (`subscriber_id`, `customer_id`, `date`) acelera la reconstrucción de sesiones, mientras que (`type_event`, `date` DESC) sirve a los filtros de QoE. En `sessions`, el índice único (`subscriber_id`, `customer_id`, `start_time`) es la clave de upsert que garantiza idempotencia.

Índices de events.

Índice	Propósito
(<code>subscriber_id</code> , <code>customer_id</code> , <code>date</code>) (<code>date</code> DESC)	Reconstrucción ordenada de sesiones. Ventanas de tiempo (alertas, actividad reciente).
(<code>type_event</code> , <code>date</code> DESC)	Filtros de QoE por tipo de evento.
(<code>customer_id</code> , <code>type_event</code>)	Rankings de contenido.
(<code>device_type</code>)	Desglose por plataforma.

Índices de sessions.

Índice	Propósito
(<code>subscriber_id</code>)	Usuarios únicos y perfil de usuario.
(<code>customer_id</code>)	Consultas por contenido.
(<code>device_type</code>)	Ranking de dispositivos.
(<code>start_time</code> DESC, <code>total_buffer_time</code> DESC)	Alertas de buffer reciente.
(<code>customer_id</code> , <code>reached_complete</code>)	Funnel de retención por contenido.
(<code>subscriber_id</code> , <code>customer_id</code> , <code>start_time</code>) UNIQUE	Clave de upsert (idempotencia).

Índices de content_stats.

Índice	Propósito
(<code>customer_id</code>) UNIQUE	Un documento por contenido.
(<code>total_plays</code> DESC)	Ranking por número de vistas.
(<code>avg_buffer_time</code> DESC)	Ranking QoE (peores por buffer).
(<code>genres</code> , <code>total_plays</code> DESC)	Rankings filtrados por género.
(<code>series_title</code> , <code>total_plays</code> DESC)	Agrupación por serie.

7 Ingesta y pipeline de datos

7.1 Parseo e ingesta de CSV

La lógica de parseo vive en `db/csv_ingest.py` y es compartida por el script de línea de comandos (`scripts/load_csv.py`) y el endpoint del panel de administración (`POST /api/admin/upload-csv`). Así, el script y la UI procesan los CSV de forma idéntica. El módulo realiza:

1. Casteo de tipos: cada columna del CSV se mapea al `bsonType` esperado. Los `NaN` de pandas se convierten en `None` y los enteros que vienen como `"2023.0"` se castean vía `float`.
2. Filtrado: se descarta toda fila sin alguno de los cinco campos obligatorios (no se puede construir una sesión a partir de ella).
3. Inserción por lotes de 1.000 documentos con `ordered=False`.

Tolerancia a fallos. Con `ordered=False`, si un documento falla la validación de Mongo, el lote continúa en lugar de detenerse. El conteo real de insertados se toma del resultado de cada lote, incluyendo el `nInserted` que reporta un `BulkWriteError`.

7.2 Construcción de sesiones

El pipeline de `db/pipelines/sessions.py` agrupa los eventos por par (`subscriber_id`, `customer_id`), los ordena por fecha y los parte en sub-sesiones cada vez que aparece un `START`. Por cada sub-sesión calcula el buffer total, los conteos de re-buffering y pausa, la posición máxima y el porcentaje de completitud.

```
1 for ev in evs:
2     if ev["type_event"] == "START" and current:
3         sub_sessions.append(current)
4         current = []
5     current.append(ev)
6 if current:
7     sub_sessions.append(current)
```

Listing 6: Partición en sub-sesiones por evento `START` (`db/pipelines/sessions.py`)

Cap de `completion_pct` a 100 %. Algunas filas traen `Position` en una unidad distinta a `Duration` (milisegundos frente a segundos) o posiciones residuales de sesiones previas, lo que dispara el porcentaje por encima de 100. Capear a 100 % evita que esos valores atípicos contaminen los promedios de completitud del dashboard:

```
1 completion_pct = (
2     round(min(max_position / duration * 100, 100.0), 2)
3     if duration and duration > 0 and max_position is not None
4     else None
5 )
```

Listing 7: Cálculo de completitud capado

7.3 Agregación de content_stats

El pipeline de `db/pipelines/content_stats.py` lee todas las sesiones, las agrupa por `customer_id` y produce un documento agregado por contenido: volumen, tasas del funnel, métricas de calidad y desglose por dispositivo. La escritura usa `UpdateOne(..., upsert=True)` por `customer_id`.

7.4 Idempotencia mediante upsert

Tanto sesiones como `content_stats` se escriben con *upsert* sobre su clave única. Cargar el mismo CSV dos veces **actualiza** los documentos en lugar de duplicarlos, lo que hace que el pipeline sea seguro de re-ejecutar.

8 API REST

Toda la API vive bajo el prefijo `/api`. Las respuestas son JSON; los errores retornan `{ "detail": "mensaje en español" }`. Los contratos se validan con esquemas Pydantic v2 y la documentación interactiva (Swagger) se genera automáticamente en `/docs`.

8.1 Módulo Overview

Endpoint	Descripción
GET /overview/unique-users	Total de suscriptores distintos.
GET /overview/total-plays	Total de reproducciones (= sesiones).
GET /overview/device-ranking	Sesiones agrupadas por dispositivo.
GET /overview/activity-by-hour	Sesiones por hora del día (las 24 horas).
GET /overview/top-content	Contenidos más vistos (param <code>limit</code>).
GET /overview/users-map	Una burbuja por usuario activo, ubicada en el centroide de su país + jitter determinístico.

El endpoint del mapa trunca el `subscriber_id` a 8 caracteres (no expone el identificador completo) y posiciona cada usuario en el centroide de su país más un *jitter* determinístico, ya que el dataset solo trae el código de país, no coordenadas.

8.2 Módulo QoE

Endpoint	Descripción
GET /qoe/buffer-by-content	Buffer promedio por contenido (peores primero).
GET /qoe/rebuffering-rate	Tasa de re-buffering por eventos y por sesiones.
GET /qoe/startup-time	Tiempo de inicialización (promedio, máximo y distribución).
GET /qoe/event-ranking	Ranking de tipos de evento por frecuencia.

8.3 Módulo Users

Endpoint	Descripción
GET /users/retention-funnel	Funnel: cuántas sesiones llegaron a cada cuartil.
GET /users/activity-heatmap	Sesiones por hora × día de la semana (168 puntos).
GET /users/content-completion-ranking	Contenidos más completados y más abandonados (param <code>min_plays</code>).
GET /users/user-profiles	Clasificación de usuarios por actividad.

Perfiles de usuario. Los usuarios se clasifican por el número total de eventos que generan: **Vague** (< 10), **Mid** ($10-49$) y **Heavy** (≥ 50). Los umbrales son constantes (`PROFILE_MID_MIN`, `PROFILE_HEAVY_MIN`) en `db/repositories/users.py`, y la clasificación se resuelve del lado del servidor con un `$switch` en la agregación.

8.4 Módulo Admin

Endpoint	Descripción
GET /admin/collections	Conteo de documentos por colección.
POST /admin/upload-csv	Sube un CSV y re-construye las derivadas.
GET /admin/documents/{col}	Documentos paginados (<code>page</code> , <code>page_size</code>).
PUT /admin/documents/{col}/{id}	Edición en vivo (<code>\$set</code> de campos).
DELETE /admin/documents/{col}/{id}	Borrado de un documento.

8.5 Ejemplos de petición y respuesta

Por ejemplo, el ranking de contenido devuelve un arreglo ya ordenado y proyectado:

```

1 GET /api/overview/top-content?limit=2
2
3 {
4   "content": [
5     { "title": "Liga BetPlay", "total_plays": 120,
6       "unique_viewers": 80, "content_type": "LIVE" },
7     { "title": "PARTIDO APARTE", "total_plays": 40,
8       "unique_viewers": 33, "content_type": "MOVIE" }
9   ]
10 }

```

Listing 8: Petición y respuesta de top-content

```

1 GET /api/users/retention-funnel
2
3 { "total": 1000, "firstquartile": 800, "midpoint": 600,
4   "thirdquartile": 400, "complete": 200 }

```

Listing 9: Respuesta del funnel de retención

Las consultas se resuelven en el servidor. Ningún repositorio trae documentos completos: las métricas se calculan con el *aggregation framework* de MongoDB. La clasificación de perfiles, por ejemplo, usa `$switch` para etiquetar a cada usuario sin traer datos al cliente:

```

1 "profile": {
2   "$switch": {
3     "branches": [
4       { "case": { "$lt": [ "$events", PROFILE_MID_MIN ] }, "then": "Vague" },
5       { "case": { "$lt": [ "$events", PROFILE_HEAVY_MIN ] }, "then": "Mid" },
6     ],
7     "default": "Heavy",
8   }
9 }

```

Listing 10: Clasificación de perfiles con `$switch` (db/repositories/users.py)

8.6 Manejo de errores

Código	Significado
400	Petición inválida (ID malformado, archivo no-CSV o vacío).
404	Recurso o colección no encontrada.
422	Error de validación de parámetros / cuerpo.
500	Error interno del servidor.

9 Dashboard (Frontend)

El frontend es una SPA en **React 19** servida con **Vite**. Las gráficas se construyen con **Recharts** y el mapa de usuarios con **Leaflet** (`react-leaflet`). El proxy de Vite redirige `/api`

hacia `http://localhost:8000`, por lo que no se requiere configurar CORS en desarrollo.

9.1 Páginas

Página	Contenido	Estado
Overview	KPIs, mapa de burbujas, ranking de dispositivos, actividad por hora	Listo
QoE	Buffer por contenido, tasa de re-buffering, tiempo de inicio, ranking de eventos	Listo
Usuarios	Perfiles, funnel de retención, heatmap, completitud por contenido	Listo
Admin	Carga de CSV, conteo de colecciones, edición/borrado en vivo	Listo
Alertas	Tablero de alertas operativas	En construcción

9.2 Componentes y temas

Cada página orquesta componentes presentacionales que consumen un endpoint específico (por ejemplo, `KPICard`, `UsersMap`, `RebufferingRate`, `RetentionFunnel`, `DocumentTable`). El dashboard define dos temas en `src/styles/theme.css`: **brand** (naranja Win Sports #FF6B00 sobre gris oscuro) y **premium** (naranja quemado sobre casi negro).

9.3 Mapa de usuarios: jitter determinístico

Como el dataset solo trae el código de país (no coordenadas), cada usuario se ubica en el centroide de su país más un desplazamiento (*jitter*) calculado de forma determinística a partir del hash MD5 de su `subscriber_id`. Así, un mismo usuario cae siempre en el mismo punto entre peticiones, evitando que todos se apilen exactamente sobre el centroide:

```

1 h = hashlib.md5(seed.encode()).hexdigest()
2 lat_off = (int(h[:8], 16) / 0xFFFFFFFF - 0.5) * 2 * spread
3 lon_off = (int(h[8:16], 16) / 0xFFFFFFFF - 0.5) * 2 * spread

```

Listing 11: Jitter determinístico (`db/repositories/overview.py`)

10 Sistema de alertas

El módulo de alertas (`db/repositories/alerts.py`) detecta condiciones operativas anómalas dentro de la ventana temporal de los datos cargados, clasificándolas en tres niveles de severidad:

Severidad	Tipo	Condición
Roja	<code>buffer_critico</code>	Sesiones con más de 15 s de buffer acumulado.
Amarilla	<code>rebuffer_rate</code>	Contenidos donde $> 30\%$ de las sesiones (mín. 10) tienen al menos un re-buffering.
Azul	<code>pause_storm</code>	Usuarios con más de 5 pausas en menos de 8 minutos en una misma sesión.

El endpoint `GET /api/alerts/` combina las tres familias, las ordena por severidad y marca temporal, y retorna además los totales por color y la ventana del período analizado. La lógica del backend está implementada; la página de Alertas del dashboard se encuentra en construcción.

11 Panel de administración

La pestaña **Admin** permite al equipo gestionar los datos sin tocar Mongo directamente:

- **Carga de CSV.** Sube un archivo, lo inserta en `events` reutilizando `db/csv_ingest.py`, y re-construye `sessions` y `content_stats`. Muestra un resumen: insertados, omitidos, total y cuántas derivadas quedaron.
- **Conteo de colecciones.** Tarjetas con el total y el desglose por colección, refrescadas tras cada operación.
- **Edición/borrado en vivo.** Tabla paginada donde se editan campos escalares (preservando su tipo original) o se borran documentos.

Seguridad de la capa de admin. Solo se exponen tres colecciones (`events`, `sessions`, `content_stats`); cualquier otra se rechaza con 404, de modo que el panel no puede tocar colecciones internas de Mongo. El `_id` es inmutable y los `ObjectId` se validan antes de operar (un ID malformado retorna 400).

Advertencia. El panel **aún no tiene autenticación**; asume que la API corre en una red interna. Añadir auth es un pendiente antes de exponerlo a producción. Editar una colección derivada es un cambio manual que una nueva carga de CSV sobrescribe; para cambios permanentes hay que editar `events` y re-procesar.

12 Decisiones de diseño

Change streams en vez de recalcular por request.

Con millones de eventos, agregar en tiempo real sería demasiado lento para un dashboard

interactivo. Las colecciones derivadas actúan como una caché persistente que los change streams mantienen al día.

validationAction: "warn" en vez de "error".

Durante una carga masiva, un documento con un campo inesperado no debe detener la inserción del resto. Los errores se registran sin interrumpir.

Upsert con clave única en sessions.

`subscriber_id + customer_id + start_time` garantiza idempotencia: re-cargar el mismo CSV no duplica sesiones.

Scripts separados para carga y pipeline.

`load_csv.py` solo inserta eventos; `test_pipeline.py` construye las derivadas. Esto permite iterar el pipeline sin volver a subir el CSV completo.

Parseo de CSV compartido.

El script CLI y el panel de admin usan el mismo módulo (`csv_ingest.py`), evitando divergencias entre ambos caminos de ingesta.

Degradación con gracia de los change streams.

Si el clúster no soporta change streams (Mongo *standalone*, sin réplica), la API loguea el problema y sigue funcionando con las derivadas que construye `test_pipeline.py`.

13 Escalabilidad y requisitos no funcionales

El sistema está diseñado para escalar del dataset de prueba al orden de magnitud de la operación real:

- **Sharding horizontal.** MongoDB particiona `events` por su clave natural de acceso, repartiendo carga y almacenamiento entre nodos sin cambiar la aplicación.
- **Vistas materializadas.** Las colecciones derivadas evitan agregar sobre millones de eventos en cada petición; el costo de cómputo se paga una vez en la ingesta, no en cada consulta del dashboard.
- **Agregación del lado del servidor.** Todos los repositorios usan `$group/$project/count_documents`; no se traen documentos completos al proceso de Python, acotando el uso de memoria.
- **Ingesta por lotes tolerante a fallos.** La inserción en lotes de 1.000 con `ordered=False` sostiene cargas grandes sin abortar ante documentos puntuales inválidos.
- **Privacidad.** Las respuestas públicas truncan el identificador de suscriptor; el mapa nunca expone el `subscriber_id` completo.

14 Pruebas

14.1 Backend

La suite de pytest usa **mongomock-motor** para simular MongoDB en memoria, de modo que las pruebas no dependen de un clúster real. Cubre:

- La lógica de los pipelines (construcción de sesiones y `content_stats`).
- El parseo y filtrado del CSV.
- Los repositorios de consulta.
- El contrato de todas las rutas de la API, incluido el panel de Admin.

```
1 # Backend
2 uv run pytest
3
4 # Frontend
5 cd frontend && npm run test
```

Listing 12: Ejecución de pruebas

14.2 Frontend

Las pruebas usan **Vitest** + **Testing Library** sobre **jsdom**. El setup global mockea **react-leaflet** (jsdom no monta el mapa) y **fetch**. Las pruebas se enfocan en los componentes presentacionales (estados de carga, vacío y con datos) y en la navegación del *shell*.

15 Riesgos y limitaciones

15.1 Resueltos

- **Duplicados en sessions.** Blindado con el índice único `idx_sessions_unique_key` y escrituras vía `upsert` sobre la clave compuesta. Un re-procesamiento actualiza la sesión en vez de duplicarla.
- **Consumo de memoria en consultas.** Ningún repositorio trae documentos completos: las consultas resuelven con agregaciones `$group/$project` o `count_documents`, que agregan del lado del servidor.

15.2 Abiertos / a vigilar

- `distinct("subscriber_id")` (usado al construir sesiones) tiene un tope de 16 MB de BSON. Si el número de suscriptores crece mucho, conviene migrar a `$group + $count`.

- El bundle del frontend supera los 500 kB (Leaflet + Recharts). Hoy es solo un *warning* de build; si molesta, se puede aplicar *code-splitting* con `import()` dinámico del mapa.
- El panel de Admin carece de autenticación.

16 Trabajo futuro

- Completar la página de Alertas del dashboard (el backend ya existe).
- Añadir autenticación/autorización al panel de administración.
- Editor tipado de fechas y de campos anidados en la tabla de Admin.
- Búsqueda/filtro dentro de la tabla de documentos.
- Despliegue de los change streams en un clúster réplica para sincronización en tiempo real.
- Migrar consultas sensibles a volumen (`distinct`) a agregaciones.

17 Conclusiones

WinSports demuestra cómo un modelo de datos documental de tres niveles —fuente de verdad inmutable más vistas materializadas— resuelve de forma elegante la tensión entre la captura de eventos crudos y la necesidad de consultas analíticas rápidas. La separación estricta de capas (base de datos, repositorios, API y frontend) hace que el sistema sea mantenible y extensible: cada componente puede evolucionar de forma independiente mientras respeta el contrato que lo conecta con el siguiente.

Las decisiones clave —change streams como caché persistente, upsert idempotente, parseo de CSV compartido, validación tolerante a fallos— responden directamente a las dos características del dominio: esquema flexible y alto volumen. El resultado es un sistema que, además de funcionar sobre el dataset de prueba, está diseñado para escalar al orden de magnitud que exigiría la operación real de Win Sports.

18 Glosario

Evento

Registro atómico emitido por el reproductor (`START`, `PAUSE`, `COMPLETE`, etc.). Una fila del CSV.

Sesión

Conjunto de eventos de un par usuario–contenido entre un `START` y el siguiente. Unidad de análisis de reproducción.

QoE (Quality of Experience)

Calidad percibida de la reproducción: buffer, re-buffering y tiempo de inicialización.

Funnel de retención

Proporción de sesiones que alcanzan cada hito de progreso (primer cuartil, mitad, tercer cuartil, completo).

Upsert

Operación que actualiza el documento si existe o lo inserta si no; base de la idempotencia del pipeline.

Change stream

Mecanismo de MongoDB que emite cambios en una colección, usado para sincronizar las colecciones derivadas en tiempo real.

Vista materializada

Colección derivada precalculada (`sessions`, `content_stats`) que actúa como caché de consultas.

Centroide

Coordenada (lat, lon) representativa de un país, usada como ancla del mapa de usuarios.

Jitter

Desplazamiento determinístico aplicado a un punto del mapa para que usuarios del mismo país no se solapen.

19 Referencias

- [1] MongoDB, Inc. *MongoDB Manual — Documents, Schema Validation y Change Streams*. <https://www.mongodb.com/docs/manual/>
- [2] MongoDB, Inc. *MongoDB Atlas Documentation*. <https://www.mongodb.com/docs/atlas/>
- [3] MongoDB, Inc. *Motor: Asynchronous Python Driver for MongoDB*. <https://motor.readthedocs.io/>
- [4] S. Ramírez. *FastAPI Documentation*. <https://fastapi.tiangolo.com/>
- [5] Pydantic. *Pydantic v2 Documentation*. <https://docs.pydantic.dev/>
- [6] Encode. *Uvicorn: An ASGI web server for Python*. <https://www.uvicorn.org/>
- [7] Meta Open Source. *React Documentation*. <https://react.dev/>
- [8] Evan You y contribuidores. *Vite: Next Generation Frontend Tooling*. <https://vitejs.dev/>

- [9] Recharts Group. *Recharts: A composable charting library built on React components*. <https://recharts.org/>
- [10] V. Agafonkin. *Leaflet: an open-source JavaScript library for interactive maps*. <https://leafletjs.com/>
- [11] The pandas development team. *pandas Documentation*. <https://pandas.pydata.org/docs/>
- [12] H. Krekel y el equipo de pytest. *pytest Documentation*. <https://docs.pytest.org/>
- [13] The Vitest Team. *Vitest: A Vite-native testing framework*. <https://vitest.dev/>
- [14] Proyecto WinSports. *Documentación interna del repositorio: docs/architecture.md, docs/collections.md, docs/endpoints.md, docs/frontend.md, docs/admin.md y docs/scripts.md*.

A Anexo: puesta en marcha

```
1 # 1. Dependencias
2 uv sync
3 cd frontend && npm install && cd ..
4
5 # 2. Variables de entorno (.env en la raíz)
6 #     MONGO_URI=mongodb+srv://<usuario>:<password>@<cluster>.mongodb.net/
7 #     MONGO_DB_NAME=winsports
8
9 # 3. Cargar datos
10 uv run scripts/load_csv.py --file data/archivo.csv
11 uv run scripts/test_pipeline.py                # construir derivadas
12 uv run scripts/test_pipeline.py --HARD         # borrar y reconstruir
13
14 # 4. Levantar el dashboard (dos terminales)
15 uv run uvicorn api.main:app --reload           # API    -> :8000
16 cd frontend && npm run dev                     # Front  -> :5173
```

Listing 13: Configuración y ejecución

Enlaces locales:

- Frontend: <http://localhost:5173>
- Documentación de la API (Swagger): <http://localhost:8000/docs>
- Health check: <http://localhost:8000/api/health>

B Anexo: estructura del repositorio

```
1 WinSports/  
2 |- api/  
3 |   |- main.py           # app FastAPI + lifespan + routers  
4 |   |- routes/           # overview, goe, users, alerts, admin  
5 |   +- schemas/          # modelos Pydantic de respuesta  
6 |- config/  
7 |   |- settings.py       # carga de .env (pydantic-settings)  
8 |   +- constants.py      # colecciones, centroides de paises  
9 |- db/  
10 |   |- connection.py     # connect / disconnect / get_db  
11 |   |- csv_ingest.py      # parseo e insercion de CSV (compartido)  
12 |   |- collections/      # validators + setup de cada coleccion  
13 |   |- indexes/          # definicion de indices  
14 |   |- pipelines/        # construccion de sessions y content_stats  
15 |   +- repositories/     # queries por modulo  
16 |- scripts/  
17 |   |- load_csv.py        # carga de events  
18 |   |- test_pipeline.py   # construye derivadas  
19 |   +- verify_connection.py # prueba de conexion a Atlas  
20 |- frontend/             # React + Vite (pages, components, layout)  
21 |- data/                 # CSVs de eventos de muestra  
22 |- docs/                 # documentacion tecnica  
23 +- tests/                # pytest (db/ y api/)
```

Listing 14: Árbol de carpetas (resumen)